Practical 9: MiniBank Multithreaded Account Synchronization

Problem Definition:

Implement AccountWorker (implementing Runnable) to perform concurrent deposits on a shared Account instance. Demonstrate balance loss under race conditions, inspect thread states, and resolve using synchronized deposit methods.

Code:

```
class PartB {
    public static class Account {
        private final String accountNumber;
        private long balance;

        public Account(String accountNumber, long initialBalance) {
            this.accountNumber = accountNumber;
            this.balance = initialBalance;
        }

        public String getAccountNumber() { return accountNumber; }
        public long getBalance() { return balance; }

        public void unsafeDeposit(long amount) {
            long current = balance;
            try { Thread.sleep(1); } catch (InterruptedException ignored) {}
            balance = current + amount;
        }

        public synchronized void safeDeposit(long amount) {
            balance += amount;
        }
    }

    public static class AccountWorker implements Runnable {
        private final Account account;
        private final int times;
        private final long amount;
        private final boolean useSafe;

        public AccountWorker(Account account, int times, long amount, boolean useSafe) {
            this.account = account;
            this.times = times;
            this.amount = amount;
            this.useSafe = useSafe;
        }

        @Override
        public void run() {
            for (int i = 0; i < times; i++) {
                if (useSafe) {
                    account.safeDeposit(amount);
                } else {
                    account.unsafeDeposit(amount);
                }
            }
        }
```

```
    }

    public static void main(String[] args) throws InterruptedException {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 9 - Part B: MiniBank Multithreaded Account
Synchronization");
        System.out.println();
        int numThreads = 10;
        int operationsPerThread = 100;
        long depositAmount = 10;
        long expectedTotal = numThreads * operationsPerThread * depositAmount;

        System.out.println("Configuration: " + numThreads + " Concurrent Threads | " +
operationsPerThread + " Deposits of Rs. " + depositAmount + " each");
        System.out.println("Expected Final Balance: Rs. " + expectedTotal);

        System.out.println("\nPhase 1: Unsynchronized Concurrent Deposits (Race Condition):");
        Account unsafeAcc = new Account("AC1001", 0);
        Thread[] threads1 = new Thread[numThreads];
        for (int i = 0; i < numThreads; i++) {
            threads1[i] = new Thread(new AccountWorker(unsafeAcc, operationsPerThread,
depositAmount, false), "WorkerThread-" + (i + 1));
            threads1[i].start();
        }
        for (Thread t : threads1) t.join();
        System.out.println("Unsafe Final Balance: Rs. " + unsafeAcc.getBalance() + " (Lost
Money: Rs. " + (expectedTotal - unsafeAcc.getBalance()) + ")");

        System.out.println("\nPhase 2: Synchronized Method Deposits (Thread-Safe Execution):");
        Account safeAcc = new Account("AC1002", 0);
        Thread[] threads2 = new Thread[numThreads];
        for (int i = 0; i < numThreads; i++) {
            threads2[i] = new Thread(new AccountWorker(safeAcc, operationsPerThread,
depositAmount, true), "SyncWorkerThread-" + (i + 1));
            System.out.println("  -> Starting Thread: " + threads2[i].getName() + " [State: " +
threads2[i].getState() + "]");
            threads2[i].start();
        }
        for (Thread t : threads2) t.join();
        System.out.println("Synchronized Final Balance: Rs. " + safeAcc.getBalance() + " [EXACT
MATCH - Thread Safe]");
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-9\PartB.java
RUDRA MODI || 25DCS058

Practical 9 - Part B: MiniBank Multithreaded Account Synchronization

Configuration: 10 Concurrent Threads | 100 Deposits of Rs. 10 each
Expected Final Balance: Rs. 10000

Phase 1: Unsynchronized Concurrent Deposits (Race Condition):
Unsafe Final Balance: Rs. 6820 (Lost Money: Rs. 3180)

Phase 2: Synchronized Method Deposits (Thread-Safe Execution):
  -> Starting Thread: SyncWorkerThread-1 [State: RUNNABLE]
  -> Starting Thread: SyncWorkerThread-2 [State: RUNNABLE]
Synchronized Final Balance: Rs. 10000 [EXACT MATCH - Thread Safe]
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: Why does concurrent unsynchronized deposit cause balance loss?

Answer: Concurrent deposit operations perform read-modify-write sequences on the shared balance field. Without synchronization, thread interleaving causes one thread's update to overwrite another's, resulting in lost updates.

Q2: How does the synchronized deposit method resolve the race condition?

Answer: Declaring deposit() as synchronized acquires the account's intrinsic monitor lock, allowing only one thread to mutate the balance at any instant while other threads wait safely in the BLOCKED state.